

Fiche d'explication technique débutant

Cette fiche sert à comprendre et expliquer le code du site AtypikHouse avec des mots simples. Elle est pensée pour une soutenance orale, même sans connaissances techniques en création de site internet.

1. Présentation simple du projet

AtypikHouse est un site web de réservation d'hébergements insolites.

Le projet est fictif et académique : aucun paiement réel, aucune réservation réelle et aucun email réel ne sont effectués.

Le site permet de montrer le fonctionnement d'une marketplace :

- un visiteur consulte les logements ;
- un locataire peut réserver ;
- un propriétaire peut proposer un logement ;
- un administrateur valide et contrôle la plateforme.

Phrase à dire :

> AtypikHouse est une plateforme fictive qui reproduit le fonctionnement d'un site de réservation, avec un espace voyageur, un espace propriétaire et un back-office administrateur.

2. Les technologies utilisées

Le site utilise plusieurs technologies.

Technologie	À quoi ça sert ?	Explication simple
PHP	Faire fonctionner le site côté serveur	C'est le langage principal du backend
MySQL	Stocker les données	C'est la base de données
PDO	Connecter PHP à MySQL	C'est une méthode sécurisée pour parler à la base
HTML	Structurer les pages	C'est le squelette visible des pages
CSS	Faire le design	Couleurs, tailles, espacements, responsive
JavaScript	Ajouter des interactions	Menu mobile, calendrier, animations simples

Phrase à dire :

> Le projet est développé en PHP avec une base de données MySQL. Les pages sont construites en HTML, stylisées en CSS, et quelques interactions sont faites en JavaScript.

3. Pourquoi on dit "site sur mesure" ?

Le site n'utilise pas WordPress, Laravel, Symfony, React ou Vue.

Cela veut dire que :

- l'architecture a été créée pour le projet ;
- les routes ont été écrites à la main ;
- les pages sont faites sur mesure ;
- le back-office est spécifique à AtypikHouse ;
- la logique métier est propre au projet.

Phrase à dire :

> Le site est fait sur mesure, sans CMS ni framework lourd. Cela permet de montrer une logique adaptée au projet AtypikHouse.

4. La structure du projet

Le projet est organisé en dossiers.

```
```text
app/
config/
database/
docs/
public/
routes/
storage/
```
```

Dossier `app`

Ce dossier contient le coeur du site.

On y trouve :

- les contrôleurs ;
- les modèles ;
- les vues ;
- les helpers ;
- les services.

Dossier `config`

Ce dossier contient les fichiers de configuration.

Exemples :

- configuration de la base de données ;
- configuration générale du site ;
- configuration des emails.

Dossier `database`

Ce dossier contient la structure et les données de la base.

Fichiers importants :

- `schema.sql` : crée les tables ;
- `seed.sql` : ajoute les données de démonstration.

Dossier `public`

Ce dossier est la partie accessible par le navigateur.

Il contient :

- `index.php`, le point d'entrée du site ;
- les fichiers CSS ;
- les fichiers JavaScript ;
- les images publiques.

Dossier `routes`

Ce dossier contient les URL du site.

Exemple :

- `/connexion`
- `/hebergements`
- `/admin/dashboard`

Dossier `storage`

Ce dossier sert à stocker certains fichiers internes.

Exemples :

- images envoyées par les propriétaires ;
- logs d'emails en mode démonstration.

5. Le point d'entrée du site

Le fichier principal qui reçoit les demandes est :

```
```text
public/index.php
```

Quand une personne ouvre le site, la demande passe par ce fichier.

Ce fichier :

- démarre la session ;
- charge les fonctions utiles ;
- charge automatiquement les fichiers PHP ;
- ajoute des protections de sécurité ;
- lance le routeur.

Phrase à dire :

> `public/index.php` est la porte d'entrée du site. Toutes les pages passent par lui avant d'être affichées.

## 6. Les routes

Les routes indiquent quelle action doit être lancée selon l'URL.

Le fichier des routes est :

```
```text
routes/web.php
```

Exemples :

URL	Rôle
---	---
`/`	page d'accueil
`/hebergements`	catalogue
`/connexion`	page de connexion
`/proprietaire/dashboard`	espace propriétaire
`/admin/dashboard`	back-office admin

Phrase à dire :

> Les routes sont comme le plan du site : elles indiquent quelle partie du code

répond à chaque URL.

7. L'architecture MVC

Le projet suit une architecture MVC.

MVC veut dire :

- Model ;
- View ;
- Contrôller.

Model

Un modèle gère les données.

Il communique avec la base de données.

Exemples :

- `User.php` pour les utilisateurs ;
- `Property.php` pour les logements ;
- `Booking.php` pour les réservations ;
- `Review.php` pour les avis.

Phrase simple :

> Le modèle sert à lire ou enregistrer les données dans la base.

View

Une vue affiche une page.

Elle contient le HTML visible par l'utilisateur.

Exemples :

- page d'accueil ;
- catalogue ;
- fiche logement ;
- dashboard admin.

Phrase simple :

> La vue est la partie que l'utilisateur voit à l'écran.

Controller

Un contrôleur reçoit les actions de l'utilisateur.

Exemples :

- connexion ;
- ajout d'un logement ;
- réservation ;
- validation admin.

Phrase simple :

> Le contrôleur fait le lien entre l'action de l'utilisateur, la base de données et la page à afficher.

Phrase complète à dire :

> Le site est organisé avec une architecture MVC. Les modèles gèrent les données, les vues affichent les pages, et les contrôleurs coordonnent les actions.

8. La base de données

La base de données utilisée est :

```
```text
MySQL
```

Son nom est :

```
```text
atypikhouse
```

Elle est créée dans :

```
```text
database/schema.sql
```

Elle est remplie avec des données de démonstration dans :

```
```text
database/seed.sql
```

Phrase à dire :

> La base de données s'appelle `atypikhouse`. Sa structure est créée dans `schema.sql` et les données de démonstration sont ajoutées avec `seed.sql`.

9. Les principales tables de la base

Une table est comme un tableau qui range un type d'information.

Table	Ce qu'elle stocke
users	les utilisateurs
owner_profiles	les profils propriétaires
properties	les logements
property_images	les images des logements
property_amenities	les équipements
property_availabilitys	les disponibilités
property_favorites	les favoris
property_change_requests	les demandes de modification logement
bookings	les réservations
payments	les paiements fictifs
reviews	les avis
blog_posts	les articles de blog
contact_messages	les messages de contact
audit_logs	les traces des actions importantes
password_resets	les demandes de réinitialisation de mot de passe

Phrase à dire :

> Chaque grande fonctionnalité du site correspond à une ou plusieurs tables dans la base de données.

10. Connexion à la base de données

La configuration de connexion est dans :

```
```text
config/database.php
```

Ce fichier contient :

- le serveur ;
- le port ;
- le nom de la base ;
- le nom d'utilisateur ;
- le mot de passe ;
- le charset.

La connexion réelle est faite dans :

```
```text
app/Core/Database.php
```

Le site utilise `PDO`.

Phrase à dire :

> Le fichier `config/database.php` contient les paramètres de connexion, et `Database.php` ouvre la connexion sécurisée à MySQL avec PDO.

11. PDO expliqué simplement

PDO est une fonctionnalité de PHP qui permet de parler à une base de données.

Dans ce projet, PDO sert à :

- lire les utilisateurs ;
- enregistrer les logements ;
- créer les réservations ;
- modifier les statuts ;
- afficher les données.

PDO est important car il permet d'utiliser des requêtes préparées.

Phrase à dire :

> PDO est utilisé pour communiquer avec MySQL de façon plus sécurisée.

12. Requêtes préparées

Une requête préparée protège la base contre certaines attaques.

Exemple simple :

Quand un utilisateur remplit un formulaire, il ne faut pas faire confiance directement à ce qu'il écrit.

Les requêtes préparées permettent de séparer :

- la commande SQL ;
- les valeurs saisies par l'utilisateur.

Phrase à dire :

> Les requêtes préparées évitent que les données saisies dans les formulaires puissent manipuler directement la base de données.

13. Les rôles utilisateurs

Le site possède trois rôles principaux :

Rôle technique	Rôle affiché
---	---
`tenant`	Locataire
`owner`	Propriétaire
`admin`	Administrateur

Locataire

Il peut :

- réserver ;
- ajouter des favoris ;
- consulter ses réservations ;
- laisser un avis.

Propriétaire

Il peut :

- proposer un logement ;
- gérer ses logements ;
- gérer ses disponibilités ;
- consulter les réservations reçues.

Administrateur

Il peut :

- valider les comptes ;
- valider les logements ;
- gérer les réservations ;
- modérer les avis ;
- traiter les messages ;
- consulter les logs.

Phrase à dire :

> Les rôles permettent de séparer les espaces et les permissions de chaque utilisateur.

14. Le système de connexion

Quand un utilisateur se connecte :

1. Il entre son email et son mot de passe.
2. Le site cherche l'utilisateur dans la table `users`.
3. Le mot de passe est vérifié.
4. Une session est créée.
5. L'utilisateur est redirigé vers son espace.

Les mots de passe ne sont pas stockés en clair.

Ils sont hashés.

Phrase à dire :

> Les mots de passe sont protégés : ils sont transformés avant d'être stockés en base.

15. Session

Une session permet au site de se souvenir qu'un utilisateur est connecté.

Exemple :

Quand un admin se connecte, la session permet au site de savoir qu'il a le droit d'ouvrir le back-office.

Phrase à dire :

> La session est une mémoire temporaire entre le navigateur et le serveur.

16. CSRF

CSRF signifie Cross-Site Request Forgery.

Dit simplement :

> C'est une protection contre les actions envoyées depuis une mauvaise page.

Dans le site, les formulaires importants ont un jeton CSRF.

Exemples :

- connexion ;
- réservation ;
- ajout logement ;
- validation admin ;
- suppression ;
- favoris.

Phrase à dire :

> Les actions sensibles sont protégées par un jeton CSRF pour éviter qu'une action soit lancée sans autorisation.

17. Échappement HTML

Le site utilise une fonction `e()`.

Elle sert à sécuriser l'affichage des textes.

Exemple :

Si quelqu'un écrit du code dans un formulaire, il ne doit pas être exécuté dans la page.

Phrase à dire :

> Les textes affichés sont échappés pour éviter qu'un utilisateur injecte du code dans les pages.

18. Le catalogue des logements

Le catalogue affiche uniquement les logements publiés.

Un logement peut avoir plusieurs statuts :

Statut technique	Affichage
--- ---	
`pending`	En attente
`published`	Publié
`rejected`	Refusé
`paused`	En pause
`deleted`	Supprimé

Important :

Les logements non publiés ne sont pas visibles sur le site public.

Phrase à dire :

> Le catalogue public affiche seulement les logements validés par l'administrateur.

19. Le workflow propriétaire

Quand un propriétaire ajoute un logement :

1. Il remplit le formulaire.
2. Il ajoute les informations principales.
3. Il ajoute les équipements.
4. Il ajoute une image principale.
5. Il peut ajouter d'autres images.
6. Le logement est enregistré en attente.
7. L'administrateur doit le valider.

Phrase à dire :

> Un logement proposé par un propriétaire ne devient pas public automatiquement. Il passe d'abord par une validation administrateur.

20. Modification d'un logement publié

Si un propriétaire modifie un logement déjà publié :

- la version publique ne change pas immédiatement ;
- une demande de modification est créée ;
- l'admin compare l'ancienne et la nouvelle version ;
- l'admin approuve ou refuse.

La table utilisée est :

```
```text
property_change_requests
```

Phrase à dire :

> Cela évite qu'un propriétaire change une fiche visible publiquement sans contrôle.

## ## 21. Le workflow réservation

Quand un locataire réserve :

1. Il choisit un logement publié.
2. Il choisit ses dates.
3. Le site vérifie les dates.
4. Le site vérifie la capacité.
5. Le site crée une réservation.
6. L'admin confirme.
7. Le locataire peut accéder au paiement fictif.

Table utilisée :

```
```text
bookings
```

Phrase à dire :

> La réservation est contrôlée avant d'être confirmée, ce qui permet à l'administrateur de suivre le cycle complet.

22. Le paiement fictif

Le paiement est fictif.

Il ne demande aucun numéro de carte.

Il sert seulement à montrer le parcours.

Table utilisée :

```
```text
payments
```

Phrase à dire :

> Le paiement est une simulation de démonstration. Aucun argent réel n'est traité.

## ## 23. Les favoris

Les favoris permettent à un locataire de garder un logement de côté.

Table utilisée :

```
```text
property_favorites
```

Fonctionnement :

- coeur vide : logement non favori ;
- coeur rempli : logement favori ;
- un utilisateur non connecté est redirigé vers la connexion.

Phrase à dire :

> Les favoris améliorent l'expérience utilisateur en permettant au locataire de retrouver facilement les logements qui l'intéressent.

24. Les avis

Les avis sont liés :

- à une réservation ;
- à un logement ;
- à un locataire.

Table utilisée :

```
```text
reviews
```
```

Les avis sont modérés par l'admin.

Phrase à dire :

> Un avis n'est visible publiquement qu'après validation de l'administrateur.

25. Le back-office administrateur

Le back-office est l'espace de contrôle.

L'admin peut :

- gérer les utilisateurs ;
- valider les propriétaires ;
- valider les logements ;
- gérer les réservations ;
- suivre les paiements fictifs ;
- modérer les avis ;
- gérer le blog ;
- traiter les messages ;
- consulter les logs.

Phrase à dire :

> Le back-office montre que le site n'est pas seulement une vitrine : il permet réellement d'administrer la plateforme.

26. Les logs d'audit

Les logs enregistrent les actions importantes.

Table utilisée :

```
```text
audit_logs
```
```

Exemples :

- connexion ;
- validation utilisateur ;
- validation logement ;
- changement statut réservation ;
- modération avis.

Phrase à dire :

> Les logs permettent de garder une trace des actions sensibles réalisées dans la plateforme.

27. Les emails

Le projet prévoit une structure d'envoi d'emails.

Mais par défaut, aucun vrai email n'est envoyé.

Les emails sont enregistrés dans un fichier local :

```
```text
storage/logs/mail-demo.log
```

Phrase à dire :

> Le système email est prêt à être configuré, mais en mode soutenance il reste en démonstration pour éviter tout envoi réel.

## ## 28. Les fichiers importants à connaître

Fichier	Rôle
public/index.php	point d'entrée du site
routes/web.php	liste des routes
config/database.php	configuration base de données
app/Core/Database.php	connexion MySQL avec PDO
database/schema.sql	création des tables
database/seed.sql	données de démonstration
app/Controllers	logique des actions
app/Models	accès aux données
app/Views	pages affichées
public/assets/css/main.css	design
public/assets/js/main.js	interactions JavaScript

## ## 29. Exemple complet à expliquer

Exemple : un propriétaire ajoute un logement.

1. Il va dans son espace propriétaire.
2. Il remplit le formulaire.
3. Le contrôleur reçoit les données.
4. Le modèle enregistre le logement dans MySQL.
5. Le logement prend le statut "En attente".
6. L'admin le voit dans son back-office.
7. L'admin le valide.
8. Le logement devient visible dans le catalogue.

Phrase à dire :

> Ce parcours montre la séparation des rôles : le propriétaire propose, l'administrateur contrôle, puis le visiteur peut consulter.

## ## 30. Conclusion technique simple

Tu peux dire :

> Techniquement, AtypikHouse est un site PHP sur mesure connecté à une base MySQL. Il utilise une organisation MVC, des requêtes préparées avec PDO, une séparation des rôles, des protections sur les formulaires et un back-office complet. Le projet reste fictif, mais il reproduit les principaux mécanismes d'une marketplace professionnelle.